RELATÓRIO PRÁTICO

PEDIDO-VELOZ — CLOUD DEVOPS E MICROSSERVIÇOS

Atividade acadêmica: Cloud DevOps: Orchestrating Containers and Micro Services

Este relatório descreve a organização do repositório e o procedimento geral para preparar, executar e verificar o ambiente Pedido-Veloz. Os passos apresentados devem ser executados e conferidos no ambiente local; resultados específicos não são presumidos neste documento.

Repositório: https://github.com/Trixbeadev/Pedido-Veloz

Vídeo de apresentação: https://youtu.be/DndaMGAUI_I

1. Objetivo

O objetivo prático é organizar e demonstrar um ambiente de aplicação baseado em serviços separados, com configurações para execução local e arquivos relacionados a automação, monitoramento, orquestração e infraestrutura como código.

O repositório inclui, conforme a estrutura observada no projeto, pastas para serviços de pedidos, pagamentos e estoque, além de diretórios relacionados a Kubernetes, monitoramento, Terraform e testes. A função exata de cada arquivo deve ser confirmada em seu conteúdo.

2. Organização do repositório

A estrutura principal observada no projeto é apresentada abaixo:

Diretório/arquivo	Finalidade geral
servico-pedidos/	Arquivos relacionados ao serviço de pedidos.
servico-pagamentos/	Arquivos relacionados ao serviço de pagamentos.
servico-estoque/	Arquivos relacionados ao serviço de estoque.
kubernetes/	Manifests e configurações de orquestração.
monitoring/	Arquivos relacionados ao monitoramento e à observabilidade.
terraform/	Arquivos de infraestrutura como código.
tests/	Arquivos de testes, conforme implementados.
docker-compose.yml	Definição dos serviços e configurações do ambiente local.
.env.example / .env	Exemplo e configuração local de variáveis de ambiente. O arquivo .env não deve conter segredos publicados.
requirements-dev.txt	Arquivo de dependências de desenvolvimento, conforme conteúdo do repositório.
.github/	Configurações e workflows de GitHub, se definidos.

Nota: a tabela descreve a finalidade esperada a partir dos nomes das pastas. A existência e o comportamento de recursos específicos dependem do conteúdo dos arquivos.

3. Pré-requisitos

Antes de executar o ambiente, recomenda-se verificar a instalação e o funcionamento de Docker Desktop, Git e PowerShell ou outro terminal compatível. Também é necessário ter os arquivos do repositório disponíveis localmente.

4. Obtenção do código

O repositório pode ser obtido com os seguintes comandos:

```
git clone https://github.com/Trixbeadev/Pedido-Veloz.git
```

```
cd Pedido-Veloz
```

Se o repositório já estiver clonado, basta abrir um terminal na pasta que contém o arquivo docker-compose.yml.

5. Configuração do ambiente

O projeto possui arquivos de variáveis de ambiente. Antes de iniciar os serviços, deve-se conferir o arquivo de exemplo e identificar quais valores são exigidos pelo Docker Compose e pelas aplicações.

Se a aplicação exigir um arquivo .env e houver um .env.example válido, uma cópia pode ser criada no PowerShell com:

```
Copy-Item .env.example .env
```

O comando acima só deve ser usado quando os nomes dos arquivos corresponderem aos existentes no repositório. Preencha os valores exigidos e nunca publique credenciais reais.

6. Inicialização dos serviços

Na pasta que contém o docker-compose.yml, execute:

```
docker compose up --build -d
```

O comando solicita a construção das imagens necessárias e inicia os serviços definidos no arquivo Compose. Se ocorrer erro, consulte a mensagem apresentada no terminal antes de prosseguir.

7. Verificação e logs

Para conferir o estado dos contêineres:

```
docker compose ps
```

Para acompanhar os logs dos serviços:

```
docker compose logs -f
```

A disponibilidade de cada serviço deve ser verificada por meio dos logs, das rotas implementadas e das portas mapeadas no arquivo Compose. Não se deve presumir que uma rota de raiz exista em todos os serviços.

8. Encerramento do ambiente

Para parar e remover os contêineres e a rede criada pelo Compose, execute:

```
docker compose down
```

Volumes persistentes podem continuar existindo depois desse comando. Evite a opção -v quando for necessário preservar dados locais.

9. Kubernetes

A pasta kubernetes/ contém os arquivos relacionados à implantação em Kubernetes. Para aplicá-los, é necessário ter um cluster disponível e o kubectl configurado. Antes de executar comandos de implantação, confira os nomes dos manifests, os namespaces, as imagens e as variáveis requeridas.

A presença dos manifests não comprova que os recursos foram aplicados em um cluster. Para documentar uma execução, registre os comandos utilizados e os resultados observados, por exemplo, a saída de kubectl get pods e kubectl get services, quando aplicável.

10. Monitoramento e rastreamento

O diretório monitoring/ reúne arquivos relacionados ao monitoramento, conforme sua configuração. Se o Prometheus estiver configurado e disponível, a consulta up pode ajudar a verificar o estado dos alvos configurados. Jaeger pode ser utilizado para explorar traces quando a instrumentação e o encaminhamento de

telemetria estiverem funcionando.

A disponibilidade das interfaces e os resultados devem ser confirmados pelas portas mapeadas no Docker Compose ou nos recursos do Kubernetes. OpenTelemetry depende da configuração de instrumentação e do destino dos dados.

11. Testes e automação

O diretório tests/ pode conter testes automatizados do projeto. A execução deve seguir as instruções ou os comandos definidos nos arquivos de configuração e nos próprios testes. Para CI/CD, os workflows armazenados em .github/workflows/ devem ser inspecionados para identificar os gatilhos e as etapas realmente automatizadas.

12. Infraestrutura como código

A pasta terraform/ contém arquivos relacionados à infraestrutura como código. Antes de executar qualquer provisionamento, revise os providers, as variáveis, os recursos declarados e o plano de execução. Não compartilhe arquivos de estado ou credenciais que contenham informações sensíveis.

13. Evidências recomendadas para a entrega

Para tornar o relatório prático verificável, recomenda-se anexar ou registrar evidências reais das etapas realizadas:

- Captura do comando docker compose ps com os serviços que foram iniciados.
- Trechos de logs que demonstrem a inicialização sem erros relevantes.
- Resultados de testes executados, com o comando e o resultado observado.
- Capturas das interfaces de monitoramento, caso tenham sido executadas.
- Resultados dos comandos Kubernetes, se um cluster tiver sido utilizado.
- Execuções do pipeline CI/CD, caso os workflows tenham sido acionados.

Essas evidências devem corresponder a resultados realmente observados. Não foram incluídos neste documento números de desempenho, percentuais de sucesso ou resultados de testes que não tenham sido fornecidos.

14. Considerações finais

O Pedido-Veloz reúne componentes e arquivos voltados ao estudo de microsserviços e práticas DevOps. A organização em serviços, a definição do ambiente local, os manifests de Kubernetes, os arquivos de monitoramento, os testes e os arquivos Terraform permitem estudar diferentes etapas do ciclo de desenvolvimento e operação.

A conclusão prática deve ser acompanhada pelas evidências da execução do ambiente e das ferramentas utilizadas. Este relatório descreve o procedimento de trabalho e indica quais verificações devem ser realizadas, sem presumir que etapas não confirmadas tenham sido executadas com sucesso.

Links do projeto

- Repositório: https://github.com/Trixbeadev/Pedido-Veloz
- Vídeo de apresentação: https://youtu.be/DndaMGAUI_I

Observação final: confirme os nomes reais das pastas e dos arquivos antes da entrega. Este relatório foi montado a partir das informações disponíveis e deve ser ajustado caso a estrutura local tenha sido alterada.